Building your first Electron app: a 3D website video studio

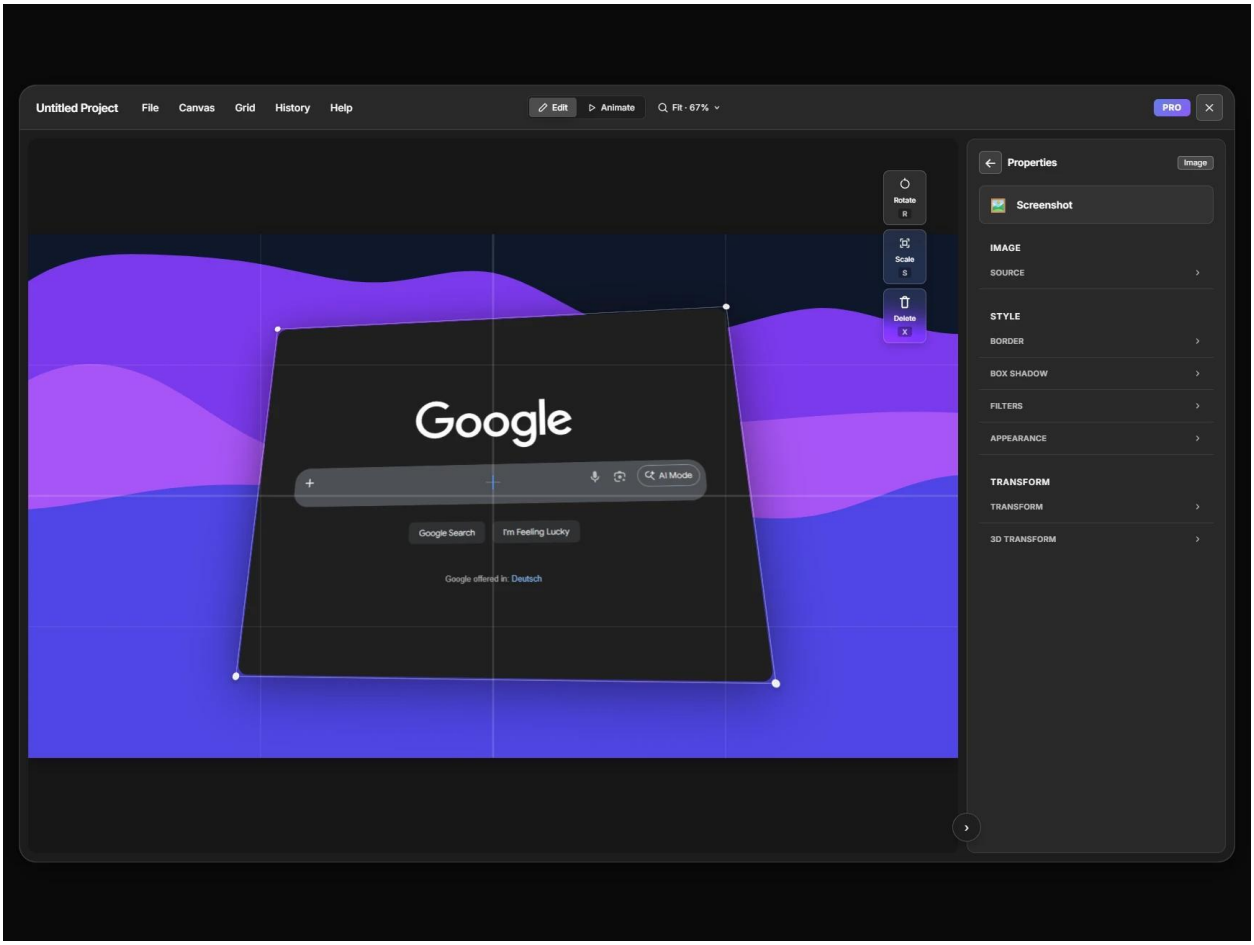
Your idea is essentially a 3D motion-design editor for real websites. Instead of recreating a website using shapes, text, and nodes, users load an actual website into a browser panel, position that panel in a 3D scene, animate it with keyframes, and export the result as an MP4 explainer video.

I think the most important decision is to build this as a small, focused animation studio first, rather than trying to build a full competitor to Blender or DaVinci Resolve immediately.

Your Canva sketch already communicates the core idea well: the 3D viewport is the stage, the website is an object on that stage, and the timeline controls how everything moves over time.

1. Turn your sketch into a concrete product

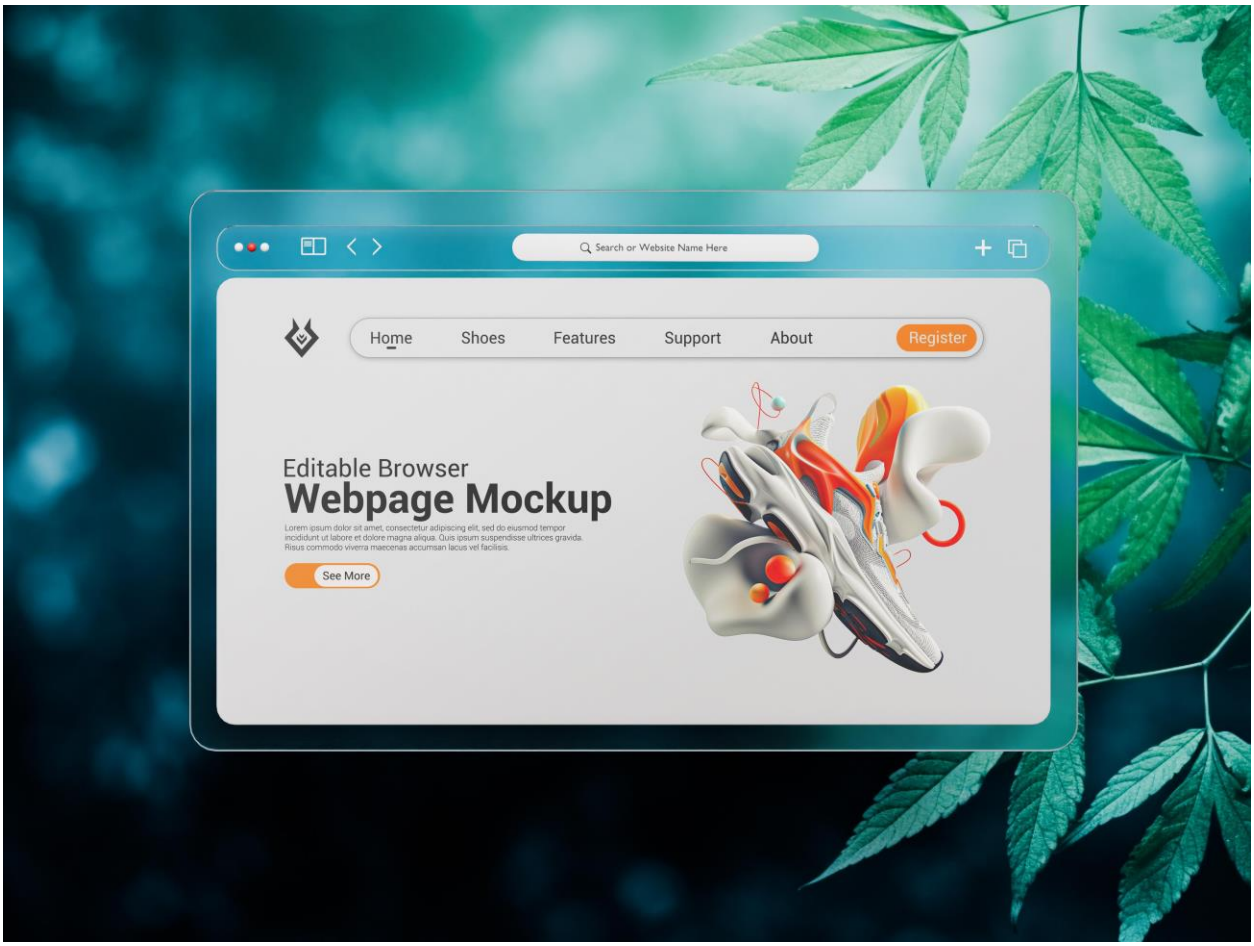
A. The 3D viewport



Your main canvas

A fixed camera looks at a 3D scene. Users can move, rotate, and scale website panels, position text and cursor objects, and preview their animation from that camera. There is no user-controlled second camera.

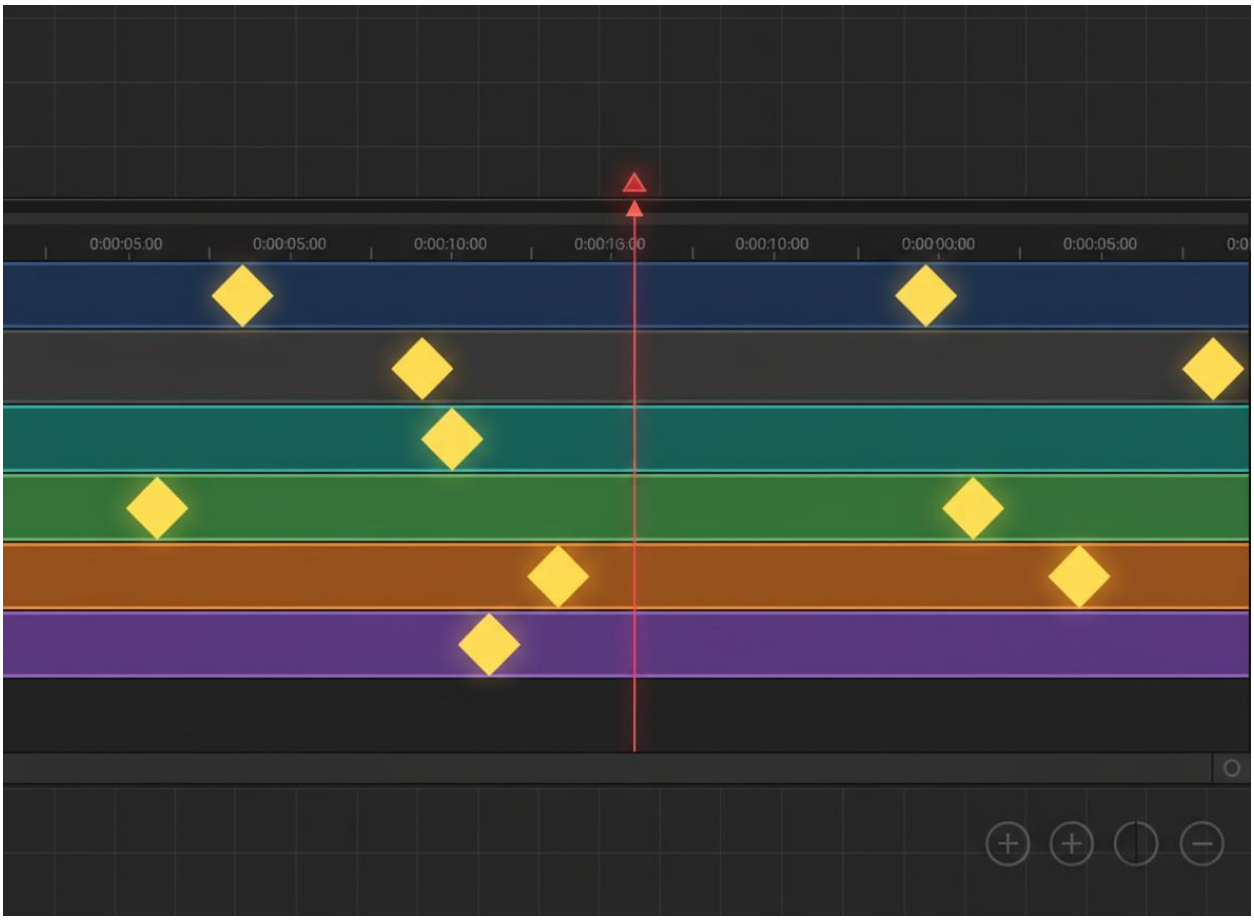
B. The live website object



Your defining feature

Users enter a URL, load a real website, and treat its visible page as a rectangular object in the scene. Its browser content can be animated without rebuilding the website UI.

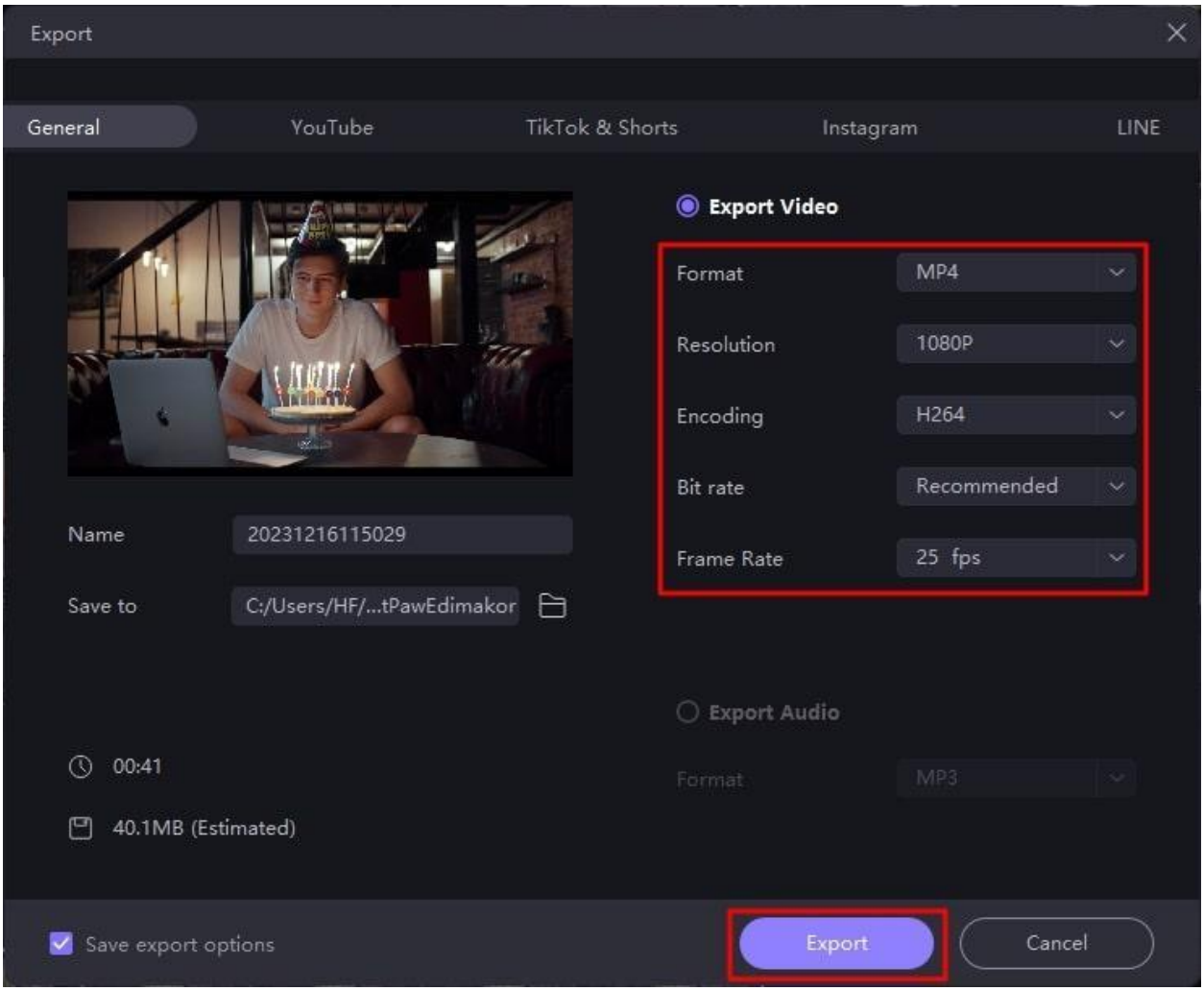
C. The animation timeline



Your control center

Users add keyframes for position, rotation, scale, cursor movement, and visibility. Scrubbing the timeline updates the 3D scene to any point in the animation.

D. The video exporter



The finished product

Render the animation at a chosen resolution and frame rate, including the website content, cursor, text, transitions, and other scene elements, then export a playable MP4.

The product's real differentiator is not the 3D viewport alone. It is the combination of real web content + controllable 3D presentation + timeline-based animation + reliable video export.

2. The biggest technical challenge: making a real website behave like a 3D object

This is the part I would design before building the rest of the interface.

An ordinary browser page is a live, interactive web document. A 3D renderer expects something it can draw as a texture on a plane. Your application needs to bridge those two worlds.

There are three possible approaches.

Approach 1: Render the browser into a texture

Best long-term target

The website runs in a real browser. Its rendered frames are captured and used as a texture on a plane in the 3D scene. The plane can then be transformed and rendered by your scene camera.

Advantages: natural perspective, depth, lighting, and compositing. Challenges: continuous frame capture, synchronization, browser security, and GPU performance.

Approach 2: Use captured website images

Capture a screenshot or sequence of screenshots, then animate those images as 3D planes.

Advantages: relatively easy to prototype and export reliably. Disadvantages: the page is not continuously live, and scrolling or changing content requires additional captures.

Approach 3: Keep the browser as a separate UI element

Display a native embedded browser above or beside the viewport and synchronize its apparent position with the 3D scene.

Advantages: excellent for initial browser interaction. Disadvantages: a normal browser view cannot simply be rotated in perspective and composited into the scene like a texture.

My recommendation: prototype Approach 2 to validate your editor and timeline, while investigating Approach 1 for the actual product. Do not assume that an Electron browser view can automatically become a live Three.js texture; that integration is a significant engineering task.

3. The technology stack I would choose

For a first release on Windows, I would use a web-based editor UI running inside Electron, with a separate, carefully isolated browser environment for websites.

Electron desktop application

Window management, file access, project lifecycle, export jobs

Editor UI

React + TypeScript

Toolbar, properties, timeline, project browser

3D engine

Three.js

Scene, camera, panels, transforms, rendering

Website runtime

Electron Chromium

Navigation, input, screenshots or frame capture

Export engine

FFmpeg

Frame encoding, audio, MP4 output

Project storage

Versioned JSON project files, local assets, cached captures and export settings.

Here is what each choice contributes:

Technology	Responsibility

Electron	Windows desktop shell and embedded Chromium
React + TypeScript	Editor panels, controls, selection and application state
Three.js	3D viewport, camera, planes, cursor and text
Zustand	Lightweight shared editor state
Custom animation engine	Keyframes, interpolation and playback
FFmpeg	Encode rendered frames and audio into MP4
Electron Forge + MSIX tooling	Package and distribute the Windows app

Use the official Electron documentation and Three.js documentation as your primary references.

A useful implementation detail: Electron's older `BrowserView` API is deprecated in favor of `WebContentsView`. Electron also documents offscreen rendering and an experimental shared-texture pathway. These are worth investigating for the website-rendering subsystem, but validate the actual integration before committing your entire architecture to it.

Electron
+1

I would not use Unity or Unreal for the first version. They add complexity without solving your hardest problem: capturing and synchronizing a live Chromium page with your editor's rendering pipeline.

4. Design the animation system before the timeline UI

The timeline should not be the source of truth for animation. It should be an interface for editing animation data.

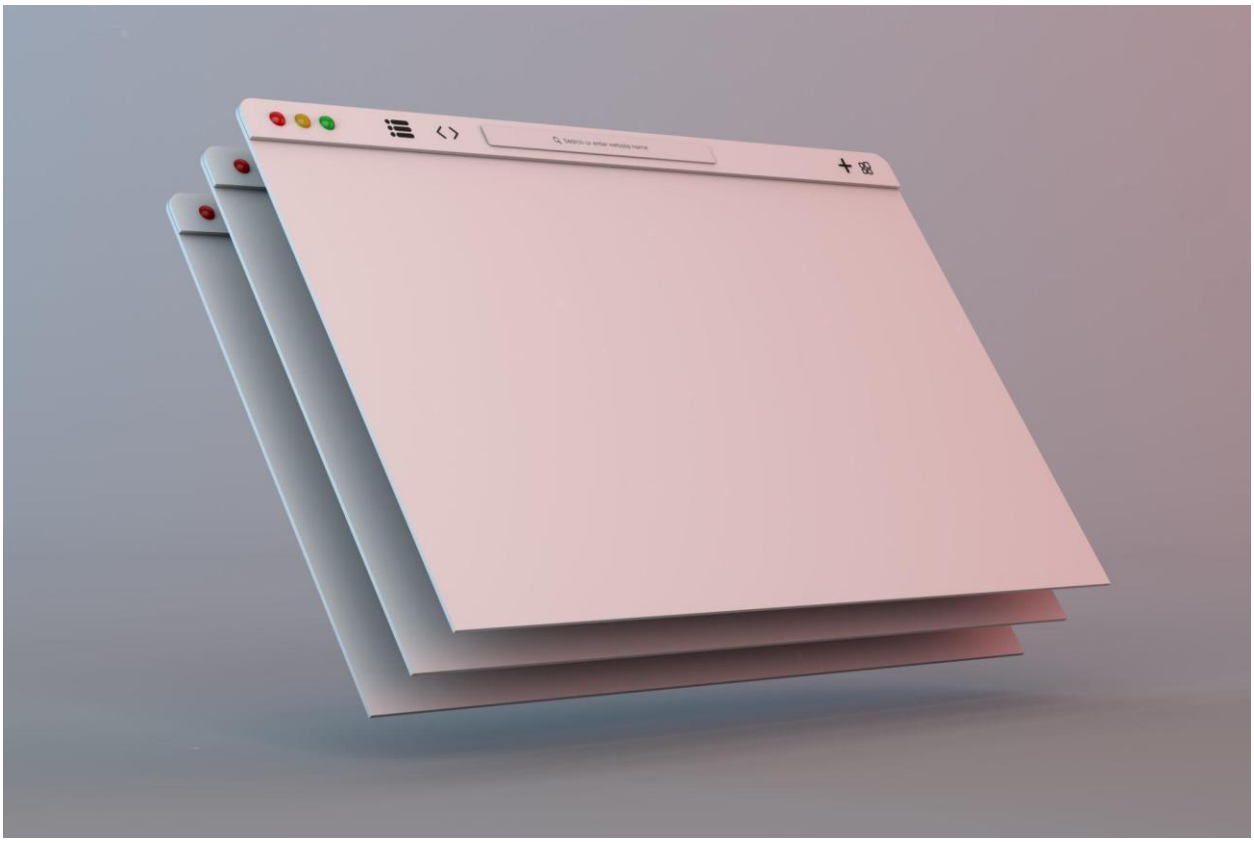
Think of every object as having a transform:

- Position: X, Y, Z
- Rotation: X, Y, Z
- Scale: X, Y, Z
- Visibility and opacity
- Optional object-specific properties

For example, your first website card might begin small and tilted, then move toward the viewer and straighten out.

EXAMPLE: WEBSITE PANEL ANIMATION

00:00



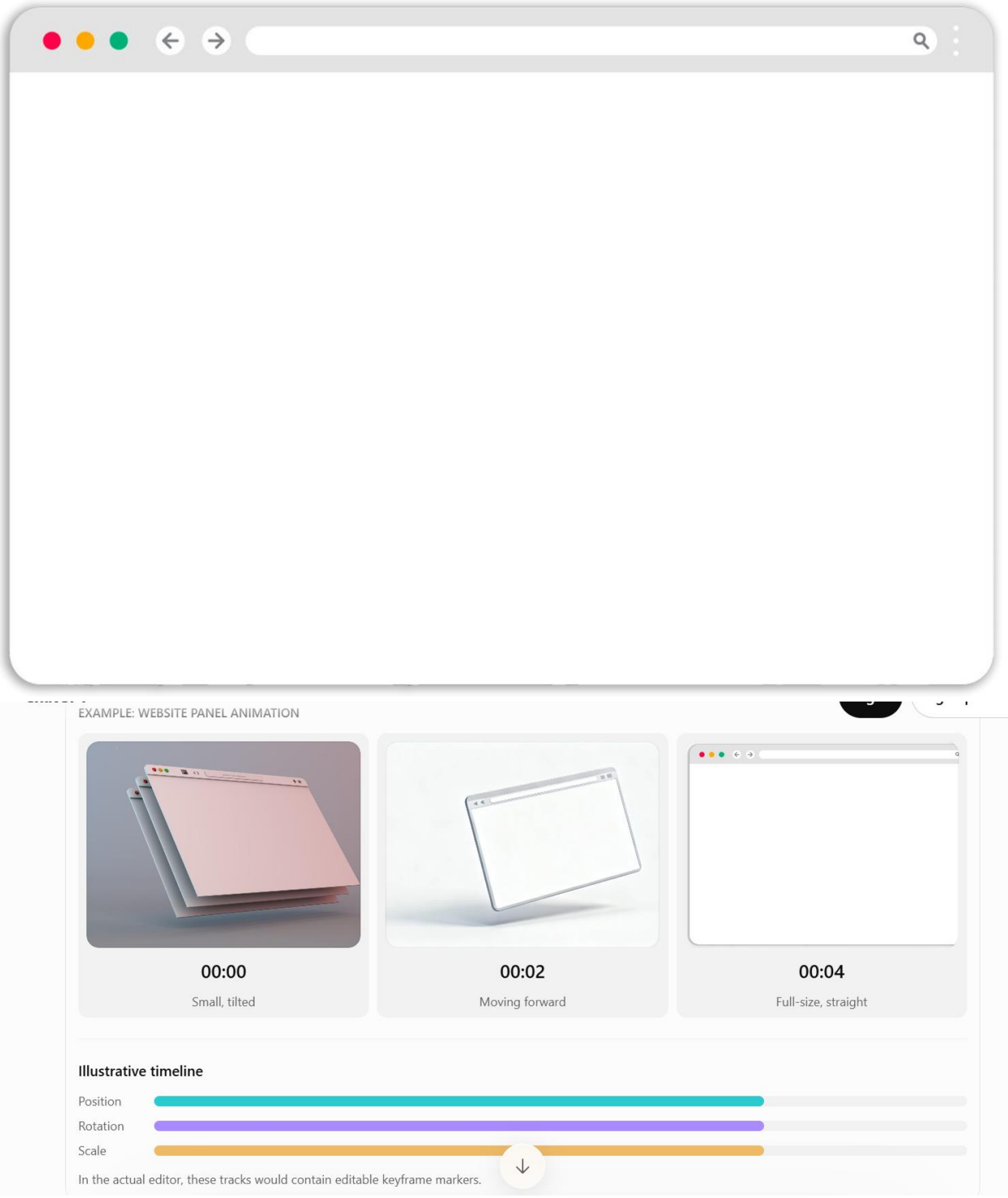
Small, tilted

00:02



Moving forward

00:04



Full-size, straight

Illustrative timeline

Position

Rotation

Scale

In the actual editor, these tracks would contain editable keyframe markers.

Recommended animation rules

1. A property can have zero or more keyframes.
2. Each keyframe contains a timeline time and a property value.
3. Between keyframes, the animation engine interpolates the value.
4. Rotation and position interpolate predictably; rotations should use an appropriate Euler or quaternion strategy to avoid unexpected flips.
5. Scale and other numeric properties can use linear or eased interpolation.
6. At any timestamp, the same project data should produce the same scene state.

That last rule is critical. The preview and final export must use the same animation evaluator.

A simplified project structure might look like this:

```
tstype Keyframe<T> = {
  time: number; // seconds
  value: T;
  easing?: "linear" | "easeInOut" | "easeOut";
};

type AnimatedProperty<T> = {
  keyframes: Keyframe<T>[];
};

type SceneObject = {
  id: string;
  type: "website" | "text" | "cursor" | "shape";
  name: string;
  position: AnimatedProperty<[number, number, number]>;
  rotation: AnimatedProperty<[number, number, number]>;
```

```
  scale: AnimatedProperty<[number, number, number]>;
};
```

This is a starting point, not a complete schema. A website object will also need a URL, viewport dimensions, browser state, and capture configuration. You will also want schema versioning, stable IDs, and project migration support.

Important design choice: store keyframes per property, not as a collection of entire scene snapshots. That allows users to animate only the scale of an object without accidentally overriding its position.

5. How the built-in cursor should work

Your cursor is more than a decorative pointer. It has two jobs:

- It must look correct in the final rendered video.
- It must interact with the actual website when the user is authoring the animation.

I would separate the cursor's visual animation from its browser input.

Visual cursor

A sprite or 3D object with keyframed screen coordinates, visibility, scale, click effects, and optional trails. This is what the camera renders into the video.

Interaction controller

Converts cursor coordinates into the website browser's viewport coordinates and sends real mouse movement, click, and scroll events to the website.

For example, suppose the website's internal viewport is 1280 × 720. A cursor position at 50% of its width and 25% of its height corresponds to:

$$x = 0.5 \times 1280 = 640$$
$$y = 0.25 \times 720 = 180$$

The challenge is mapping the 3D cursor's projected location back to the website's 2D coordinates. You can solve this using raycasting against the website plane and converting the intersection into normalized UV coordinates.

That gives you a useful pipeline:

1. The cursor is positioned in the 3D scene.
2. The camera projects it onto the website plane.
3. The intersection is mapped to website coordinates.
4. Browser events are dispatched to the embedded page.
5. The visual cursor and interaction events stay synchronized during playback and export.

For the first version, support basic pointer movement, clicks, and scrolling. Defer drag-and-drop, complex multi-touch behavior, and arbitrary keyboard interaction until the simpler system works reliably.

6. The video export pipeline

This is the second major architectural decision. An editor that plays smoothly at interactive speed can still produce unreliable exports if rendering depends on real-time playback.

Do not simply record the application window. That would capture the editor panels, timeline, and toolbar instead of just the final composition.

Instead, create a dedicated render pipeline.

1

Load the project

Read the scene, animation data, website capture settings, and export configuration.

2

Evaluate a precise timestamp

Calculate all animated properties at the exact time of each output frame.

3

Update the website image

Use a captured or live browser frame corresponding to the intended animation time.

4

Render the scene

Render the 3D camera output, including the website plane, cursor, text, and effects.

5

Encode the frames

Feed frames into FFmpeg to produce a constant-frame-rate MP4, with optional audio.

Choose your first export target

I'd start with:

Setting	Initial target
Resolution	1280 × 720
Frame rate	30 FPS
Format	MP4, H.264
Duration	5–15 seconds
Audio	Optional; can come later
Scene	One website panel, cursor, and basic text

At 30 FPS, a 10-second video contains 300 frames. The exporter should render those frames based on timestamps, not on how quickly a user's computer happens to draw them.

For frame n, use:

$$t_n = \frac{n}{30}$$

This approach makes object animation deterministic. It does not automatically make the website itself deterministic: a live page may load new content, run animations, or respond to time and network events. For the first release, you should explicitly decide how browser content is captured, frozen, or replayed during export.

I would use FFmpeg as a separate encoding process and keep rendering independent of the editor's UI. That makes it easier to add progress reporting, cancellation, error recovery, and background exports later.

7. What should actually be in version 1?

The biggest risk is trying to ship every feature in your sketch before proving that the central workflow works.

Here is the first version I would build.

Must have

- One 3D viewport and one fixed camera.
- One website panel loading a user-entered URL.
- Move, rotate, and scale controls.
- Timeline with playhead, playback, and keyframe creation.
- Position, rotation, and scale animation.
- A custom animated cursor with basic click events.
- Save and reopen local projects.
- MP4 export at one supported resolution and frame rate.

Add once the core works

- Multiple website panels.
- Animated text and simple 3D shapes.
- Camera-independent scene editing with selection and snapping.
- Fade transitions and click highlights.
- Undo and redo.
- Custom cursor styles and scrolling.

- Audio tracks and simple sound effects.

Leave for later releases

- Full nonlinear video editing.
- Advanced lighting and shadows.
- Templates and reusable animation presets.
- Cloud projects and collaboration.
- Plugins, scripting, and complex effects.
- Multiple simultaneous live website panels if they create performance problems.

I would also avoid implementing a general-purpose 3D modeling system. Your product should have a small library of specialized objects: website panels, cursor objects, text, basic shapes, and perhaps image panels. That constraint is an advantage because it lets you make the editing experience much simpler than Blender's.

8. A practical development roadmap

The schedule below is a planning estimate for one developer working consistently, not a guarantee. The website-to-texture integration could change the schedule substantially.

Phase 1

1–2 weeks

Build the editor shell

Create the Electron app, React layout, viewport, object selection, transform gizmos, and property inspector. Use placeholder planes instead of live websites.

Phase 2

2–4 weeks

Prove the browser integration

Load a website in an isolated browser process. Test frame capture, input routing, scrolling, navigation, and whether the captured frames can be displayed on a Three.js plane.

Phase 3

2–4 weeks

Implement animation and timeline

Build the project data model, keyframe engine, interpolation, timeline editing, playhead, and save/load support.

Phase 4

2–4 weeks

Make video export reliable

Render timestamped frames, add cursor animation, encode MP4, and test playback, dropped frames, memory use, and browser-content consistency.

Phase 5

1–2 weeks

Prepare the first release

Add crash handling, app branding, project validation, packaging, installer testing, Microsoft Store metadata, and a small set of test projects.

A rough total is 8–16 weeks for a narrow first version if you are already comfortable with TypeScript, Electron, and 3D fundamentals. If you're learning these technologies at the same time, or the browser-capture integration proves difficult, plan for a longer development cycle.

Your first proof of concept

Before implementing the full timeline, build a single test that proves this entire chain:

1. Load a real website.
2. Capture its visual output.
3. Display that output on a Three.js plane.
4. Rotate, move, and scale the plane in the scene.
5. Animate the plane between two timestamps.
6. Export a short MP4 with the website and cursor visible.

If you can do that reliably, you have validated the hardest technical assumption behind the product. Everything else becomes a more conventional editor-development problem.

9. Plan the project architecture so it can grow

I would separate the application into modules early, even if you are the only developer.

```
textwebsite-video-studio/
├── src/
│   ├── main/                  # Electron main process
│   ├── preload/               # Safe IPC bridge
│   ├── renderer/
│   │   ├── components/        # Toolbar, panels, dialogs
│   │   ├── viewport/          # Three.js scene and camera
│   │   ├── timeline/          # Tracks, keyframes, playhead
│   │   ├── inspector/         # Object properties
│   │   └── state/             # Editor state
│   ├── engine/
│   │   ├── animation/         # Keyframe evaluation
│   │   ├── scene/             # Scene objects
│   │   └── cursor/            # Cursor and input mapping
│   ├── browser/
│   │   ├── sessions/          # Website navigation
│   │   ├── capture/           # Browser frame capture
│   │   └── input/             # Mouse and keyboard routing
│   ├── export/
│   │   ├── renderer/          # Timestamp-based rendering
│   │   └── encoder/           # FFmpeg integration
│   └── project/
│       ├── schema/            # Project format
│       ├── storage/           # Save and load
│       └── migration/         # Format upgrades
├── tests/
└── package.json
```

This is a suggested structure, not something you need to create all at once. Start with the viewport and browser-capture modules, then add the other folders as those features become real.

Two engineering rules I would enforce from day one

1. Keep the editor separate from the websites it loads.

A website is untrusted content. Keep Node.js integration disabled for remote pages, use context isolation and sandboxing where appropriate, validate navigation and IPC messages, and do not expose unrestricted filesystem access to websites. The website browser should never have the same privileges as your editor.

2. Keep animation data separate from rendering.

The animation engine should answer, "Where is this object at 2.5 seconds?" without depending on whether the viewport is currently playing or paused. The viewport and exporter should both consume that result.

These two choices will save you from expensive architectural rewrites later.

10. Publishing the app to the Microsoft Store

You can absolutely distribute an Electron app through the Microsoft Store as an MSIX package. Microsoft provides an official Electron-to-MSIX packaging guide .

Microsoft Learn
+1

Your release process should look like this:

1. Build and test the Electron app locally.
2. Package the Windows application using Electron Forge and an MSIX packaging tool, or another supported packaging workflow.
3. Test the MSIX installation, launching, saving, reopening, and uninstalling on a clean Windows environment.
4. Create a Microsoft Partner Center developer account and reserve your application name.
5. Configure the package identity and manifest using your Store registration details.
6. Submit the package, screenshots, description, age rating, and other required listing information for certification.

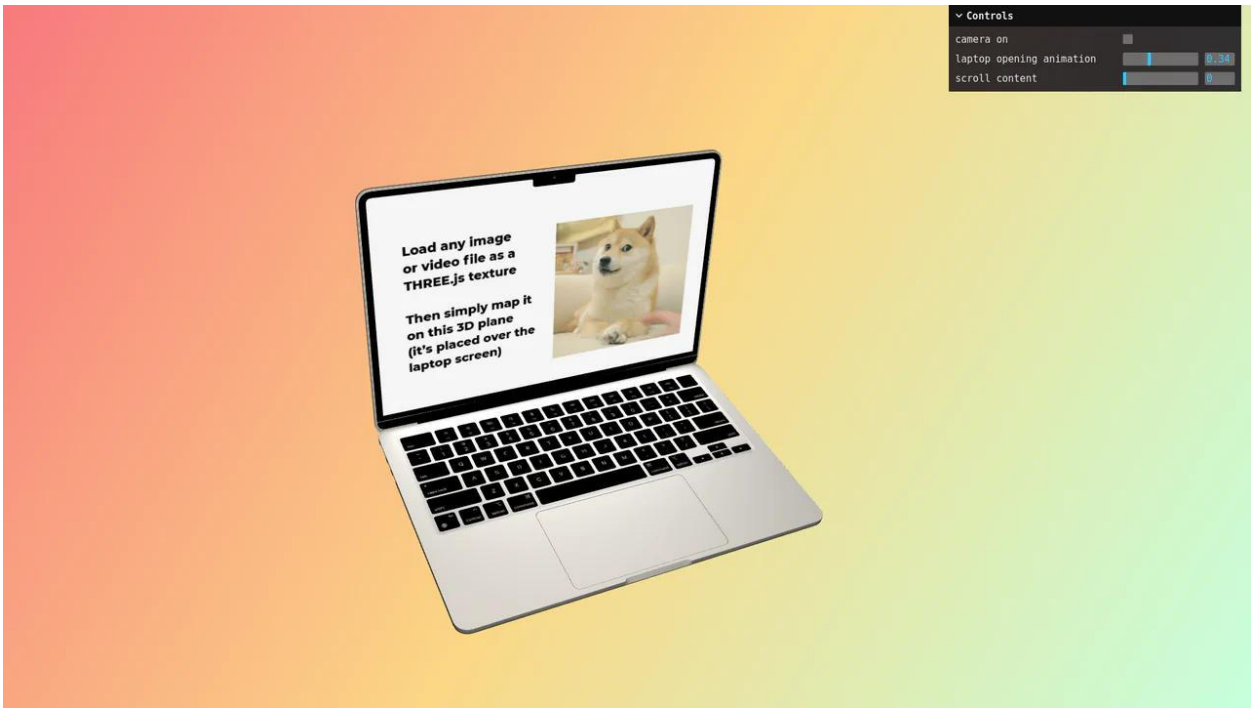
Don't leave MSIX compatibility until the final week. Your app's use of Chromium, native dependencies, file access, and the FFmpeg process must all be tested in the packaged application.

For a first release, I'd target Windows x64 initially, unless you have a specific need to support ARM64 at launch. Add additional architectures once your dependencies and rendering pipeline have been verified on them.

11. What I would do first if I were building this

I'd build three small prototypes in this order:

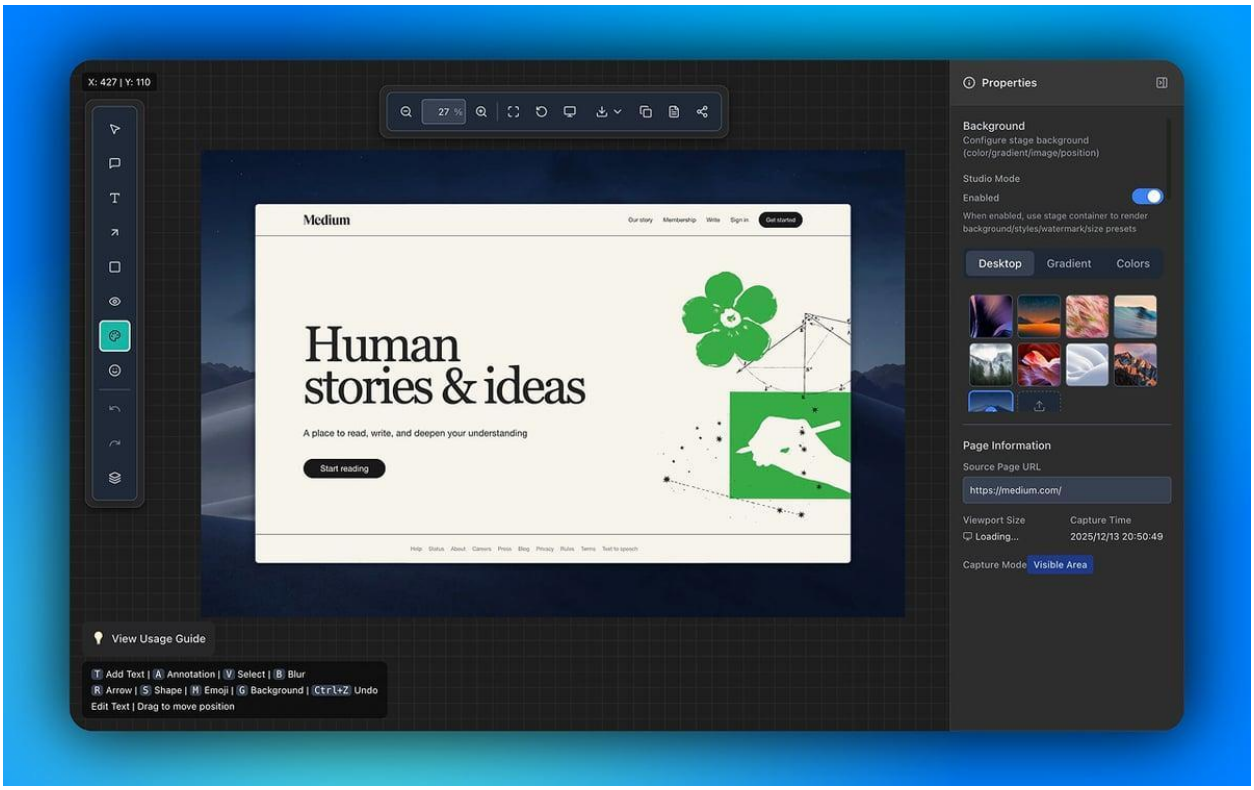
Prototype A — The scene



One plane, one fixed camera, and transform controls. Animate position and rotation with two manually defined keyframes.

Goal: prove the scene feels right.

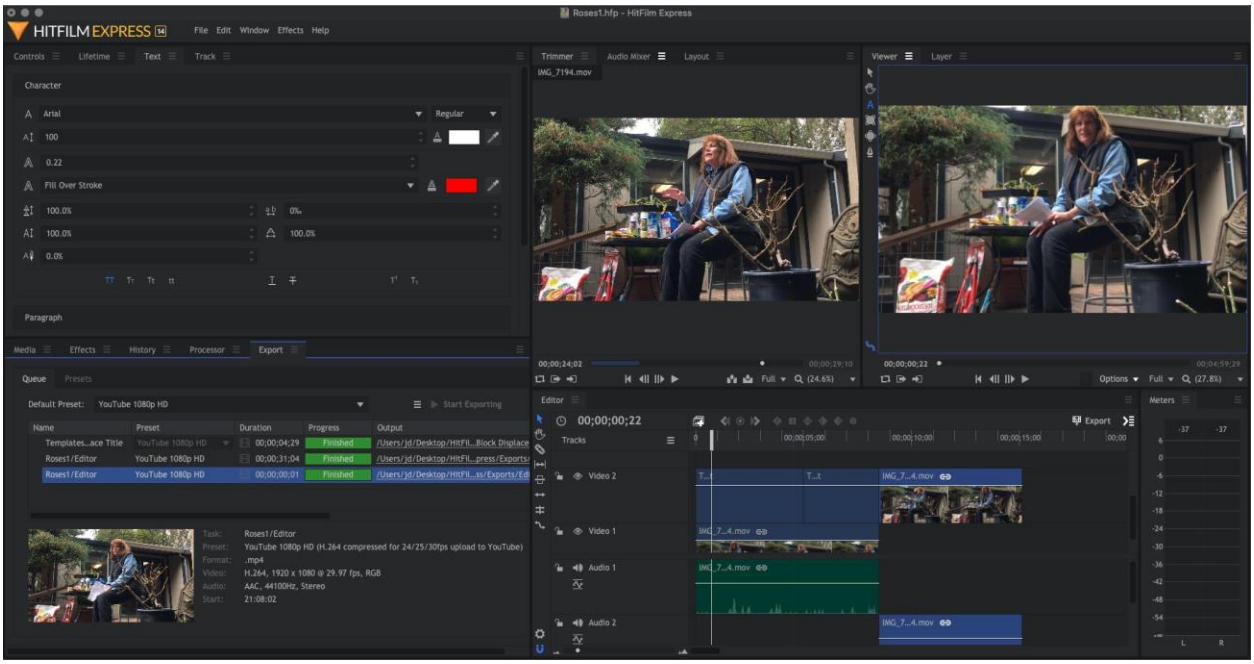
Prototype B — The live website



Load a website, capture its output, map it to the plane, and send basic cursor input back to the browser.

Goal: prove your defining feature is feasible.

Prototype C — The export



Animate the plane, render 150 frames for a five-second clip at 30 FPS, and encode them into MP4.

Goal: prove the complete output pipeline.

Only after all three prototypes work together would I invest heavily in reproducing the full interface in your sketch.

One additional product decision: I'd make the editor's camera fixed by default, as you described, but allow users to change the scene's composition through object transforms. That keeps the learning curve low while still allowing dramatic perspective, depth, and motion.

Let's tailor the build plan to you

The most useful next step is to determine how much of the technology you already know, because that affects whether we should start with the UI, the 3D engine, or the browser-capture experiment.

1. How much development experience do you have?

I'm a beginner or still learning programming
some I know some JavaScript or TypeScript and web development

I'm comfortable building full applications

2. What should we solve first?

website Getting a live website into the 3D viewport
Building the editor interface and timeline
Making reliable MP4 exports
Setting up the complete project architecture

3. How should the first prototype handle website content?

investigate Investigate live browser-to-texture rendering immediately
Start with screenshots, then upgrade to live rendering
Choose whichever approach is most practical

4. Who will build it?

solo Just me
Me and a small team
I'll hire developers
Create my development plan